- be divided into modules for each task using:
  - procedures
  - functions
- be fully commented using your programming language's commenting feature.

Table 8.26 Programming languages commenting features

Example comments	Language
`#Python uses hash to start a comment for every line`	Python
`'Visual Basic uses a single quote to start a comment for` `' every line`	Visual Basic

`// Java uses a double slash to start a single line comment`
`and`
`/* to start multiple line comments`
`and to end them`
`*/`
	Java

Activity 8.21

Check your last two programs and make sure they could be maintained by another programmer. Swap your program listing with another student and check that they can understand it.

8.2 Arrays

An **array** is a data structure containing several elements of the same data type; these elements can be accessed using the same identifier name. The position of each element in an array is identified using the array's **index**.

8.2.1 One- and Two-dimensional arrays

Arrays are used to store multiple data items in a uniformly accessible manner; all the data items use the same identifier and each data item can be accessed separately by the use of an index. In this way, lists of items can be stored, searched and put into an order. For example, a list of names can be ordered alphabetically, or a list of temperatures can be searched to find a particular value.

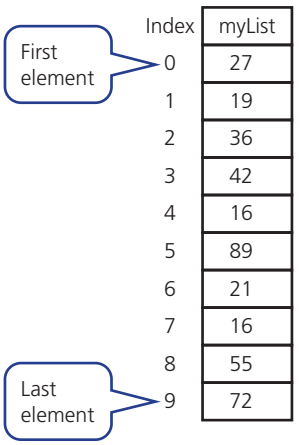
The first element of an array can have an index of zero or one. However, most programming languages automatically set the first index of an array to zero.

Arrays can be one-dimensional or multi-dimensional. One-dimensional and two-dimensional arrays are included the IGCSE Computer Science syllabus.

8.2.2 Declaring and populating arrays with iteration

One-dimensional arrays

A one-dimensional array can be referred to as a list. Here is an example of a list with 10 elements in it where the first element has an index of zero.



▲ **Figure 8.10** A one-dimensional array

When a one-dimensional array is declared in pseudocode:

- the name of the array
- the first index value
- the last index value
- and the data type

are included.

For example, to declare a new array called **MyList**:

```
DECLARE MyList : ARRAY[0:9] OF INTEGER
```

Each position in the array can be populated in an array by defining the value at each index position. For instance, we can add the number 27 to the fourth position in the array **MyList** as follows:

```
MyList[4] ← 27
```

To populate the entire array instead we can use a loop:

```
OUTPUT "Enter these 10 values in order 27, 19, 36, 42, 16, 89, 21, 16, 55, 72"
FOR Counter ← 0 TO 9
    OUTPUT "Enter next value "
    INPUT MyList[Counter]
NEXT Counter
```

Notice that in this code we have used the variable **Counter** as the array index.

We can display the data that lies in a particular location in an array as follows:

```
OUTPUT MyList[1]
```

This would display the value 19.

Activity 8.22

In your chosen programming language write a short program to declare and populate the array **MyList**, as shown in Figure 8.10, using a **FOR** loop.

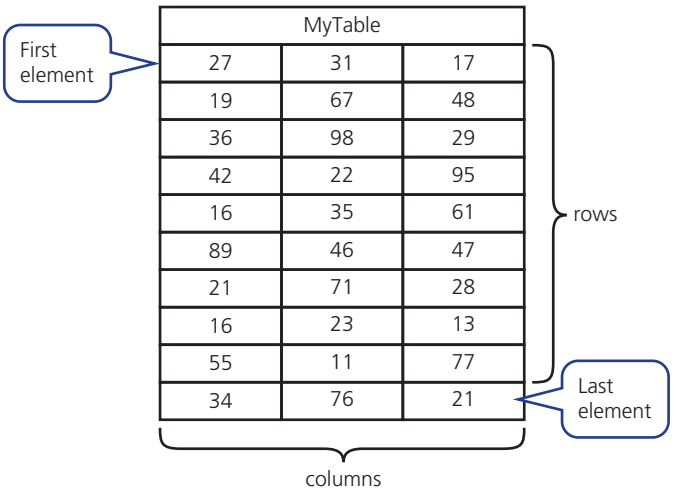
Arrays can also be populated as they are declared.

▼ **Table 8.27** Array population

Array	Language
`myList = [27, 19, 36, 42, 16, 89, 21, 16, 55, 72]`	Python
`Dim myList = New Integer() {27, 19, 36, 42, 16, 89, 21, 16, 55, 72}`	Visual Basic
`int[] myList = {27, 19, 36, 42, 16, 89, 21, 16, 55, 72};`	Java

Two-dimensional arrays

A two-dimensional array can be referred to as a table, with rows and columns. Here is an example of a table with 10 rows and 3 columns, which contains 30 elements. The first element is located at position 0,0.



▲ **Figure 8.11** A two-dimensional array

When a two-dimensional array is declared in pseudocode:

- the first index value for rows
- the last index value for rows
- the first index value for columns
- the last index value for columns
- and the data type

are included.

For example:

```
DECLARE MyTable : ARRAY[0:9,0:2] OF INTEGER
```

The declared array can then be populated using a loop, just like for one-dimensional arrays – however this time there need to be two nested loops, one for each index:

```
OUTPUT "Enter these values in order 27, 19, 36, 42, 16, 89, 21, 16, 55, 34"
OUTPUT "Enter these values in order 31, 67, 98, 22, 35, 46, 71, 23, 11, 76"
OUTPUT "Enter these values in order 17, 48, 29, 95, 61, 47, 28, 13, 77, 21"
FOR ColumnCounter ← 0 TO 2
    FOR RowCounter ← 0 TO 9
        OUTPUT "Enter next value "
        INPUT MyTable[RowCounter, ColumnCounter]
    NEXT RowCounter
NEXT ColumnCounter
```

We can display the data that lies in a particular location in a two-dimensional array as follows:

```
OUTPUT MyList[2,1]
```

This would display the value 98.

Advice

Notes on Python and arrays

Instead of arrays, Python uses another object called a list. The differences you need to know are:

- a list can contain different data types whereas arrays must all hold the same type of data
- to achieve the same structure as a two-dimensional array, Python embeds lists within another.

Two-dimensional arrays are populated in each language as follows:

Two-dimensional arrays	Language
`MyTable = [[27, 31, 17], [19, 67, 48],[36, 98, 29],[42, 22, 95], [16, 35, 61], [89, 46, 47], [21, 71, 28], [16, 23, 13], [55, 11, 77] [34, 76, 21]]`	Python
`Dim MyTable = New Integer(8, 2) {{27, 31, 17}, {19, 67, 48}, {36, 98, 29}, {42, 22, 95}, {16, 35, 61}, {89, 46, 47}, {21, 71, 28}, {16, 23, 13}, {55, 11, 77}, (34, 76, 21)}`	Visual Basic
`int[][] MyTable = {{27, 31, 17}, {19, 67, 48}, {36, 98, 29}, {42, 22, 95}, {16, 35, 61}, {89, 46, 47}, {21, 71, 28}, {16, 23, 13}, {55, 11, 77}, {34, 76, 21}};`	Java

Activity 8.23

In your chosen programming language, write a program to declare and populate the array **MyTable**, as shown in Figure 8.11, using a nested **FOR** loop.

8.3 File handling

8.3.1 Purpose of storing data in a file

Computer programs store data that will be required again in a **file**. While any data stored in RAM will be lost when the computer is switched off, when data is saved to a file it is stored permanently. Data stored in a file can thus be accessed by the same program at a later date or accessed by another program. Data stored in a file can also be sent to be used on other computer(s). The storage of data in files is one of the most used features of programming.

8.3.2 Using files

Every file is identified by its filename. In this section, we are going to look at how to read and write a line of text or a single item of data to a file.

Here are examples of writing a line of text to a file and reading the line of text back from the file. The pseudocode algorithm has comments to explain each stage of the task.

pseudocode

```
DECLARE TextLine : STRING  // variables are declared as normal
DECLARE MyFile : STRING
MyFile ← "MyText.txt"
// writing the line of text to the file
OPEN MyFile FOR WRITE  // opens file for writing
    OUTPUT "Please enter a line of text"
    INPUT TextLine
    WRITEFILE, TextLine  // writes a line of text to the file
CLOSEFILE(MyFile)  // closes the file
// reading the line of text from the file
OUTPUT "The file contains this line of text:"
OPEN MyFile FOR READ  // opens file for reading
    READFILE, TextLine  // reads a line of text from the file
    OUTPUT TextLine
CLOSEFILE(MyFile)  // closes the file
```